

React Forms Validation Hands-On Lab — Solution

Building a Registration Form ("mailregisterapp") with Controlled Components

Objectives

- Explain React Forms validation
- Identify the differences between a React form and an HTML form
- Explain controlled components
- Identify various React Form input controls
- Explain how to handle React Forms
- Explain about submitting forms in React

In this hands-on lab, you will learn how to:

- Implement React forms validation
- Use various input controls like textbox, button, and textarea

Prerequisites

- Node.js
- NPM
- Visual Studio Code

Task

Create a React app named "mailregisterapp" with a component named Register that renders a form accepting Name, Email, and Password. The fields must be validated as follows: Name must have at least 5 characters, Email must contain both '@' and '.', and Password must have at least 8 characters. Validation must be implemented through the form's change and submit event handlers.

Solution — Step by Step

Step 1: Create the React project "mailregisterapp"

Open a terminal and run the following command to scaffold the project:

```
npx create-react-app mailregisterapp
```

Once the setup finishes, navigate into the project folder:

```
cd mailregisterapp
```

Step 2: Open the application in VS Code

```
code .
```

Step 3: Create the Register component

Under the src folder, add a new file named register.js and start the class component:

```
import React, { Component } from 'react';

class Register extends Component {
  // constructor, handleChange(), validate(),
  // handleSubmit() and render() are added in the
  // following steps
}

export default Register;
```

Step 4: Initialize the form as controlled components

Add a constructor that initializes state for the field values, the field-level errors, and a submission status. Storing each input's value in state (and setting it back via the value attribute) is what makes these controlled components, as opposed to a plain HTML form where the DOM itself owns the input values:

```
constructor(props) {
  super(props);
  this.state = {
    name: '',
    email: '',
    password: '',
    errors: {
      name: '',
      email: '',
      password: ''
    },
    submitted: false
  };
  this.handleChange = this.handleChange.bind(this);
  this.handleSubmit = this.handleSubmit.bind(this);
}
```

Step 5: Define the validate() helper

Add a validate() method that checks a single field's value against its rule and returns an error message (or an empty string if the value is valid):

```
validate(field, value) {
  switch (field) {
    case 'name':
      return value.length < 5
        ? 'Name must be at least 5 characters long'
        : '';
    case 'email':
      return !value.includes('@') || !value.includes('.')
        ? 'Email must contain "@" and "."'
        : '';
    case 'password':
      return value.length < 8
```

```

        ? 'Password must be at least 8 characters long'
        : '';
    default:
        return '';
    }
}

```

Step 6: Implement handleChange() (the onChange event handler)

Add a handleChange() method that updates the corresponding field in state on every keystroke and immediately re-validates that field, giving the user real-time feedback:

```

handleChange(event) {
  const { name, value } = event.target;
  const errorMessage = this.validate(name, value);

  this.setState(prevState => ({
    [name]: value,
    errors: {
      ...prevState.errors,
      [name]: errorMessage
    }
  }));
}

```

Step 7: Implement handleSubmit() (the onSubmit event handler)

Add a handleSubmit() method that prevents the default HTML form submission (a page reload), re-validates every field, and only marks the form as submitted if there are no errors:

```

handleSubmit(event) {
  event.preventDefault();

  const errors = {
    name: this.validate('name', this.state.name),
    email: this.validate('email', this.state.email),
    password: this.validate('password', this.state.password)
  };

  const hasErrors = Object.values(errors).some(
    message => message !== ''
  );

  this.setState({ errors, submitted: !hasErrors });
}

```

Step 8: Implement render() with the form and input controls

Add the render() method with a text input for Name, a text/email input for Email, and a password input for Password, each wired to state via value and onChange, plus a submit button:

```

render() {
  const { name, email, password, errors, submitted } = this.state;

  return (
    <div className="register-form">
      <h1>Register</h1>
      <form onSubmit={this.handleSubmit}>
        <div>
          <label>Name</label>
          <input
            type="text"
            name="name"
            value={name}
            onChange={this.handleChange}
          />
          {errors.name && <p className="error">{errors.name}</p>}
        </div>

        <div>
          <label>Email</label>
          <input
            type="text"
            name="email"
            value={email}
            onChange={this.handleChange}
          />
          {errors.email && <p className="error">{errors.email}</p>}
        </div>

        <div>
          <label>Password</label>
          <input
            type="password"
            name="password"
            value={password}
            onChange={this.handleChange}
          />
          {errors.password && (
            <p className="error">{errors.password}</p>
          )}
        </div>

        <button type="submit">Register</button>
      </form>

      {submitted && (
        <p className="success">Registration successful!</p>
      )}
    </div>
  );
}

```

```
}
```

Complete register.js

Putting all the pieces together, the complete register.js file looks like this:

```
import React, { Component } from 'react';

class Register extends Component {
  constructor(props) {
    super(props);
    this.state = {
      name: '',
      email: '',
      password: '',
      errors: {
        name: '',
        email: '',
        password: ''
      },
      submitted: false
    };
    this.handleChange = this.handleChange.bind(this);
    this.handleSubmit = this.handleSubmit.bind(this);
  }

  validate(field, value) {
    switch (field) {
      case 'name':
        return value.length < 5
          ? 'Name must be at least 5 characters long'
          : '';
      case 'email':
        return !value.includes('@') || !value.includes('.')
          ? 'Email must contain "@" and "."'
          : '';
      case 'password':
        return value.length < 8
          ? 'Password must be at least 8 characters long'
          : '';
      default:
        return '';
    }
  }

  handleChange(event) {
    const { name, value } = event.target;
    const errorMessage = this.validate(name, value);

    this.setState(prevState => ({
```

```

        [name]: value,
        errors: {
            ...prevState.errors,
            [name]: errorMessage
        }
    }));
}

handleSubmit(event) {
    event.preventDefault();

    const errors = {
        name: this.validate('name', this.state.name),
        email: this.validate('email', this.state.email),
        password: this.validate('password', this.state.password)
    };

    const hasErrors = Object.values(errors).some(
        message => message !== ''
    );

    this.setState({ errors, submitted: !hasErrors });
}

render() {
    const { name, email, password, errors, submitted } = this.state;

    return (
        <div className="register-form">
            <h1>Register</h1>
            <form onSubmit={this.handleSubmit}>
                <div>
                    <label>Name</label>
                    <input
                        type="text"
                        name="name"
                        value={name}
                        onChange={this.handleChange}
                    />
                    {errors.name && <p className="error">{errors.name}</p>}
                </div>

                <div>
                    <label>Email</label>
                    <input
                        type="text"
                        name="email"
                        value={email}
                        onChange={this.handleChange}
                    />

```

```

        {errors.email && <p className="error">{errors.email}</p>}
      </div>

      <div>
        <label>Password</label>
        <input
          type="password"
          name="password"
          value={password}
          onChange={this.handleChange}
        />
        {errors.password && (
          <p className="error">{errors.password}</p>
        )}
      </div>

      <button type="submit">Register</button>
    </form>

    {submitted && (
      <p className="success">Registration successful!</p>
    )}
  </div>
);
}
}

export default Register;

```

Step 9: Add the Register component to App.js

Update App.js to import and render the Register component:

```

import React from 'react';
import './App.css';
import Register from './register';

function App() {
  return (
    <div className="App">
      <Register />
    </div>
  );
}

export default App;

```

Step 10: Build and run the application

```
npm start
```

Open a browser window and type the following in the address bar:

```
http://localhost:3000
```

Typing in each field triggers `handleChange()`, which validates that field immediately and shows an inline error message when the rule isn't met (for example, a Name shorter than 5 characters, an Email missing '@' or '.', or a Password shorter than 8 characters). Clicking Register triggers `handleSubmit()`, which re-validates all three fields, prevents the default page reload, and only shows the "Registration successful!" message once every field passes validation.

Result

A React application named "mailregisterapp" was successfully created with a Register component that implements a controlled form for Name, Email, and Password. Field-level validation runs on every change through `handleChange()`, and full validation runs again on submit through `handleSubmit()`, which also prevents the default form submission behavior. The component was added to `App.js` and verified by running the application locally and confirming the validation rules and success message behave as expected at `http://localhost:3000`.